PRG 107 – Python Programming Kristiania University of Applied Sciences Lec 11

By Huamin Ren

Huamin.ren@kristiania.no

Outline

1. Data visualization
   - Simple line plots
   - Simple scatter plots
   - Visualizing errors
   - Density and contour plots

Outline

1. ***Data visualization***
   - Simple line plots
   - Simple scatter plots
   - Visualizing errors
   - Density and contour plots

1.1 Introduction to matplotlib

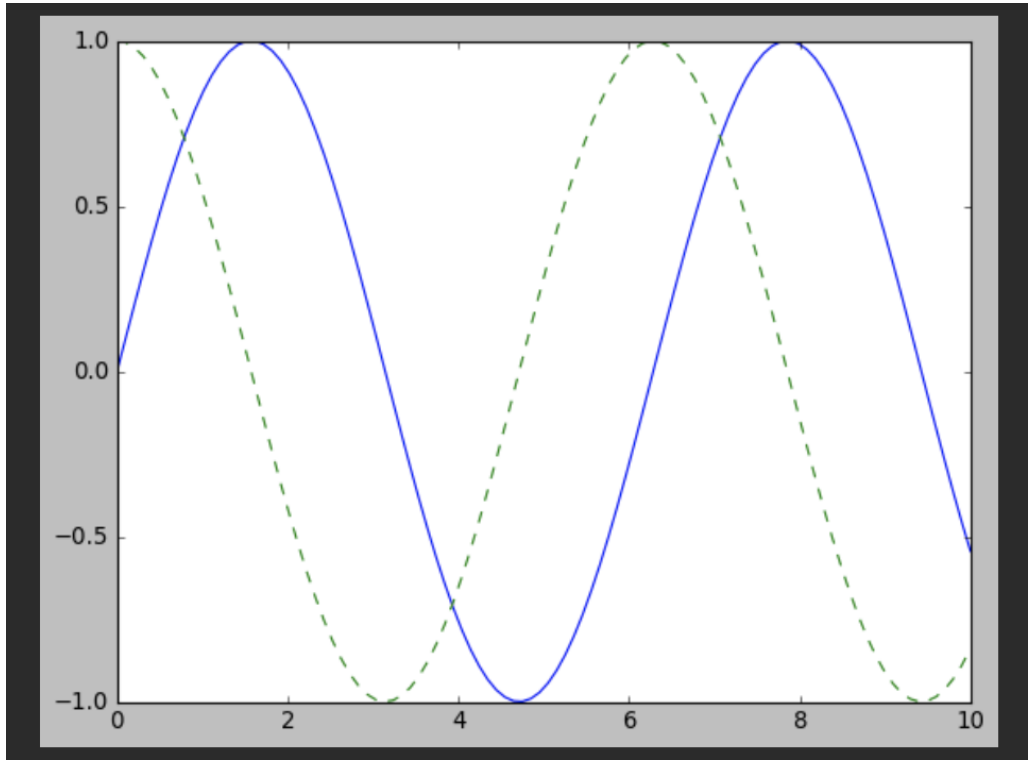
- Matplotlib has proven to be an incredibly useful and popular visualization tool
- One of major complaints: Matplotlib predated Pandas by more than a decade, and thus is not designed for use with Pandas DataFrames
- An answer to these problems is Seaborn

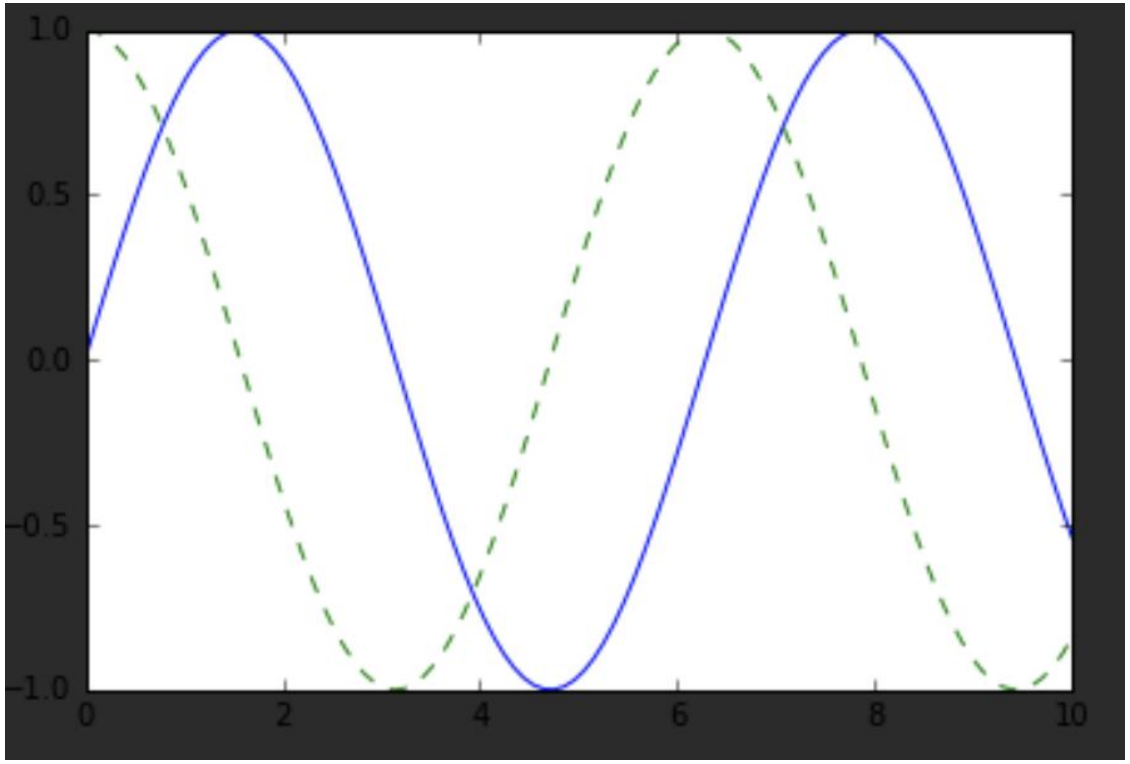
How to import?

- import matplotlib.pyplot as plt

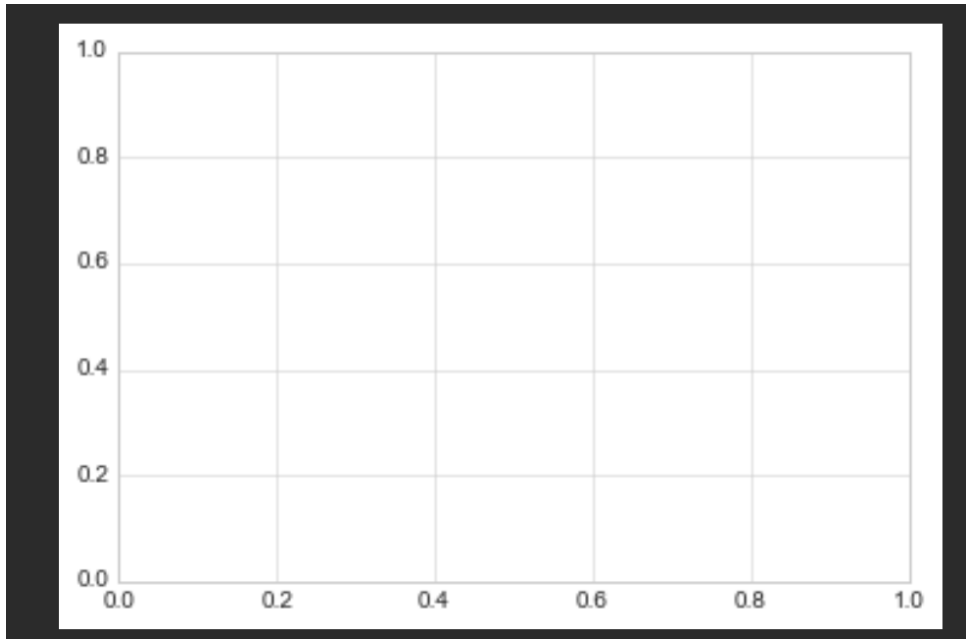
- %**matplotlib** inline

After running this command (it needs to be done only once per kernel/session), any cell within the notebook that creates a plot will embed a PNG image of the resulting graphic


1.0
0.5
0.0
−0.5
−1.0
0
2
4
6
8
10


1.0
0.5
0.0
−0.5
−1.0
0
2
4
6
8
10

1.2 Simple line plots



- The figure

An instance of the class plt.Figure, can be thought of as a single container that contains all the objects representing axes, graphics, text, and labels.

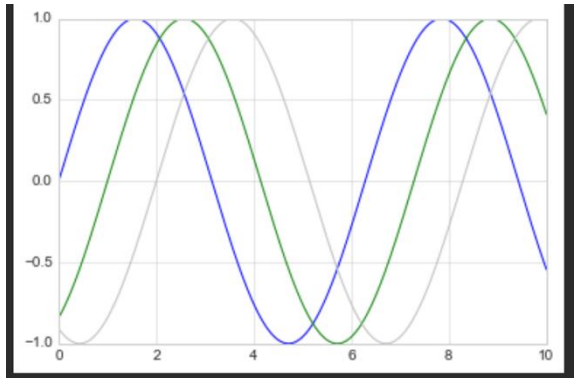
- The axes

An instance of the class plt.Axes: a bounding box with ticks and labels, which will eventually contain the plot elements that make up our visualization.

- Adjusting the Plot: Line Colors and Styles

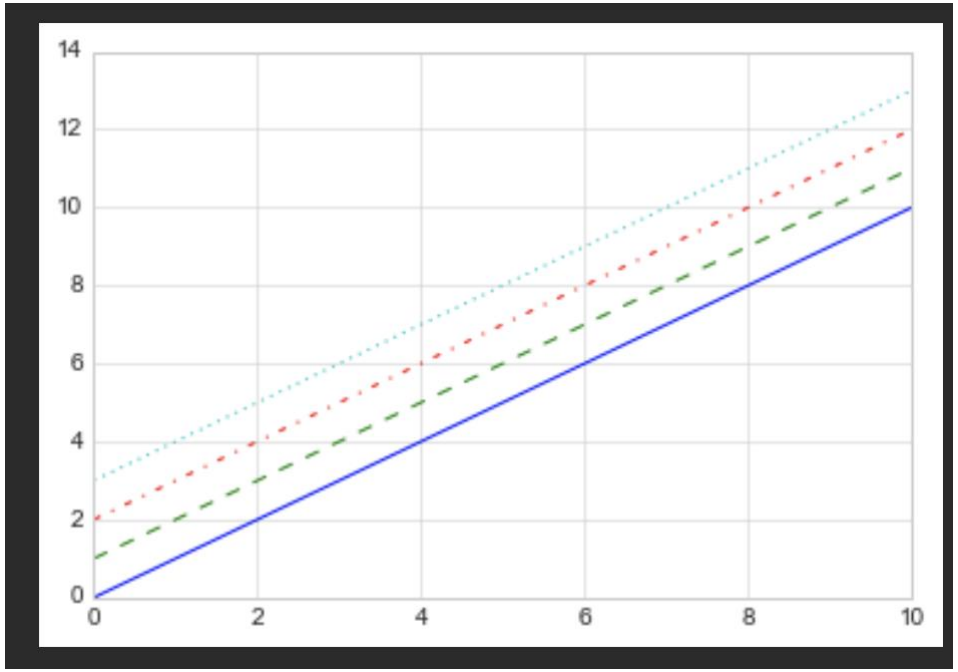
The plt.plot() function takes additional arguments to adjust the color (use the color keyword - a string argument representing virtually any imaginable color).

If no color is specified, Matplotlib will automatically cycle through a set of default colors for multiple lines.



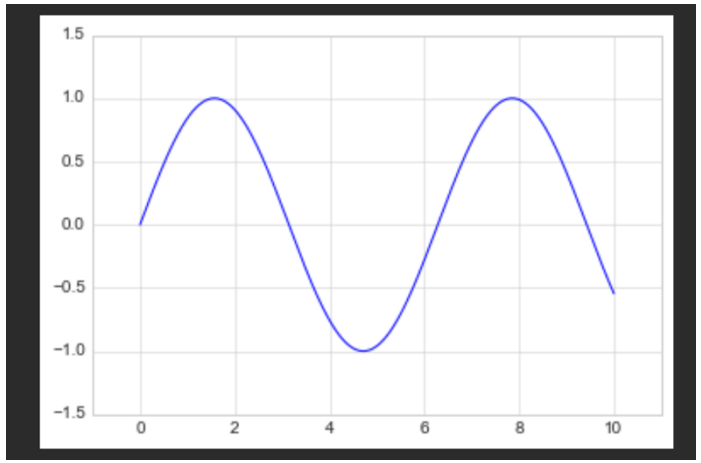
- Adjusting the Plot: Line Colors and Styles

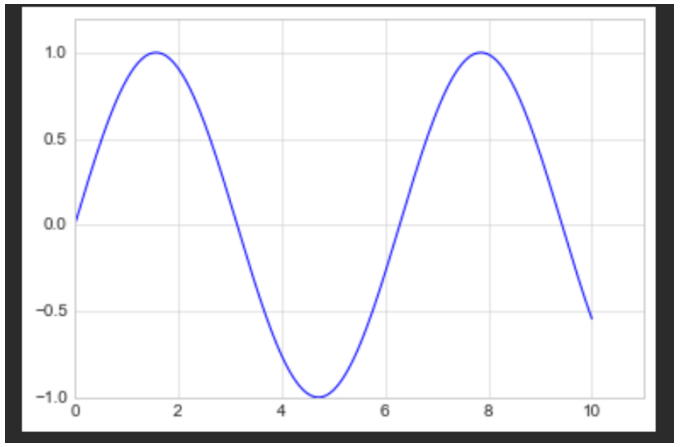
The line style can be adjusted using the linestyle keyword



- Adjusting the Plot: Axes Limits

Matplotlib does a decent job of choosing default axes limits for your plot, but sometimes it's nice to have finer control. The most basic way to adjust axis limits is to use the plt.xlim() and plt.ylim() methods

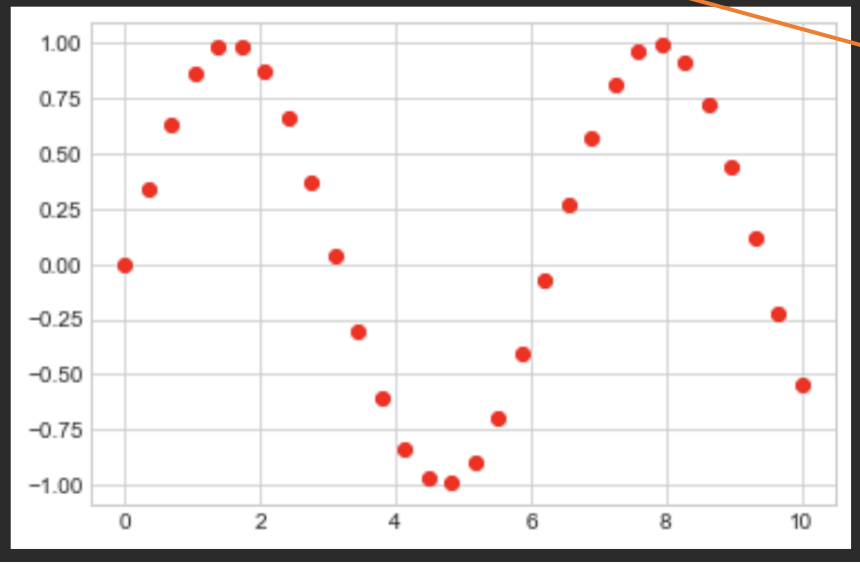




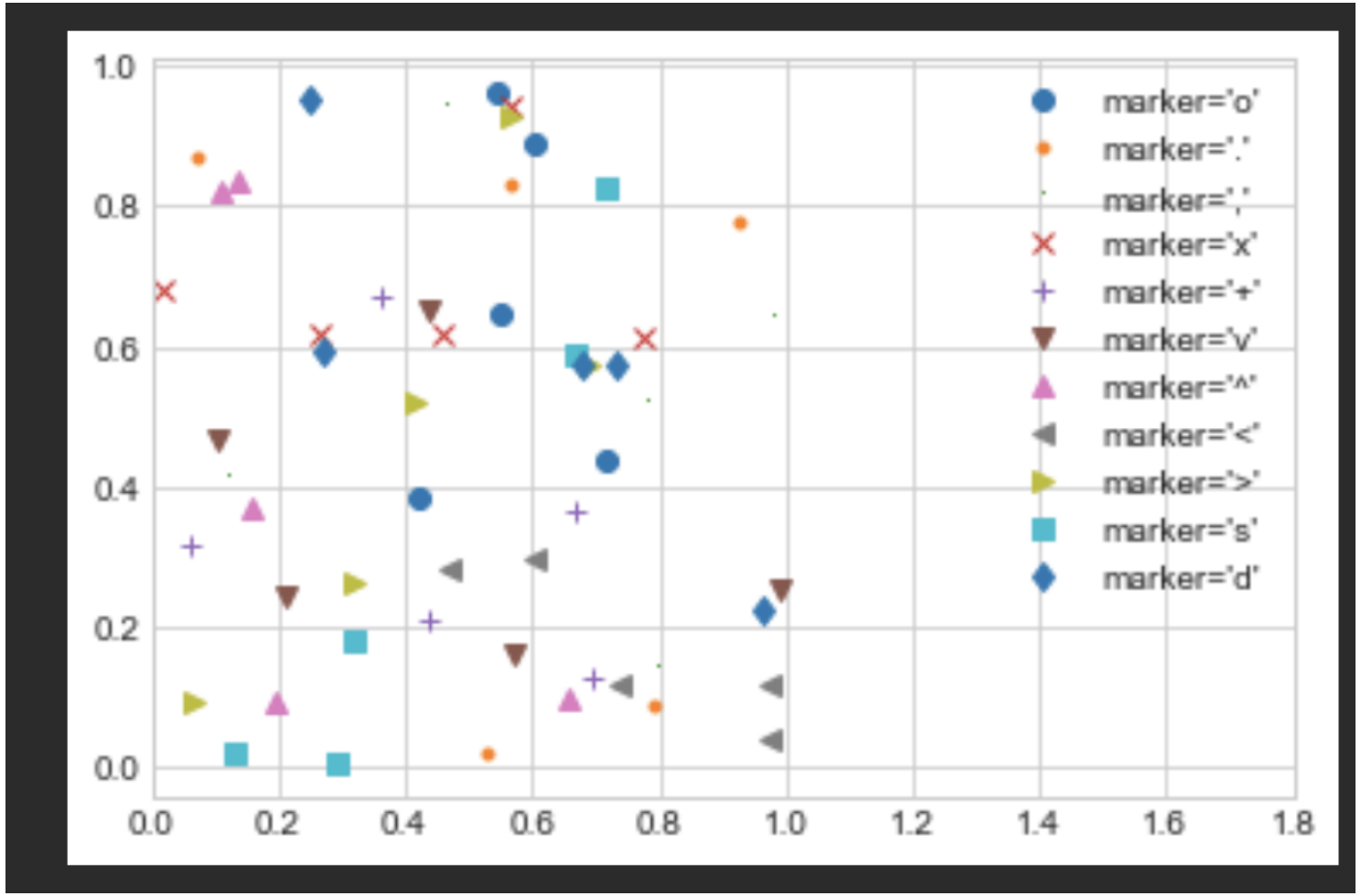
1.3 Simple scatter plots

- Scatter plots: instead of points being joined by line segments, here the points are represented individually with a dot, circle, or other shape.

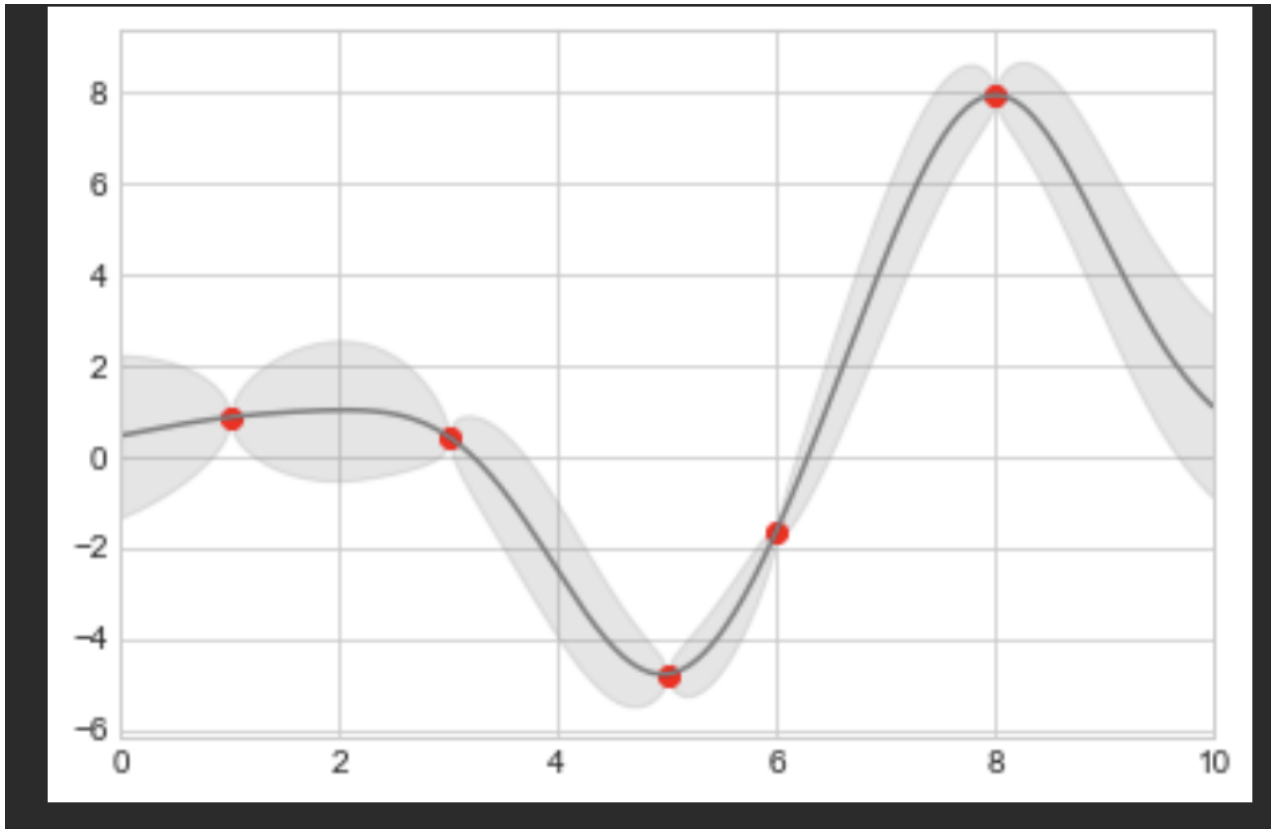
```
plt.plot(x, y, 'o', color='r');
```



represents the type of symbol used for the plotting

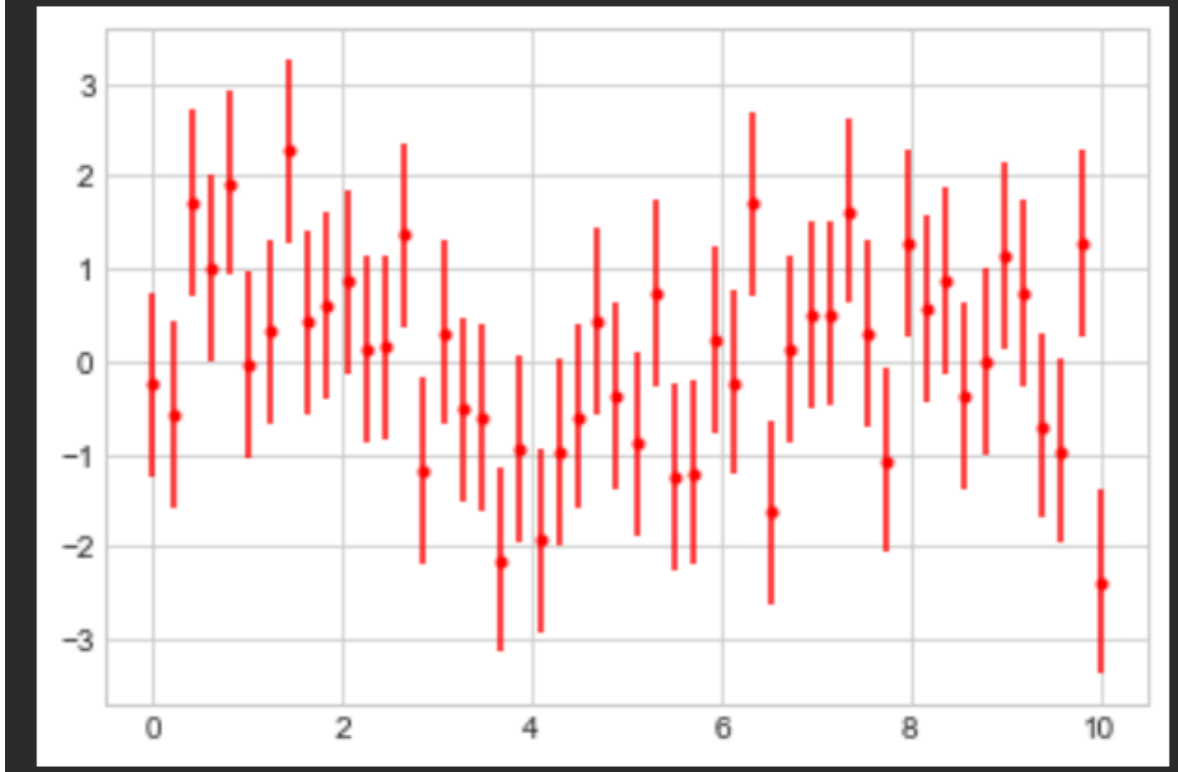

1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
1.8
marker='o'
marker='.'
marker=','
marker='x'
marker='+'
marker='v'
marker='^'
marker='<'
marker='>'
marker='s'
marker='d'

1.4 Visualize errors



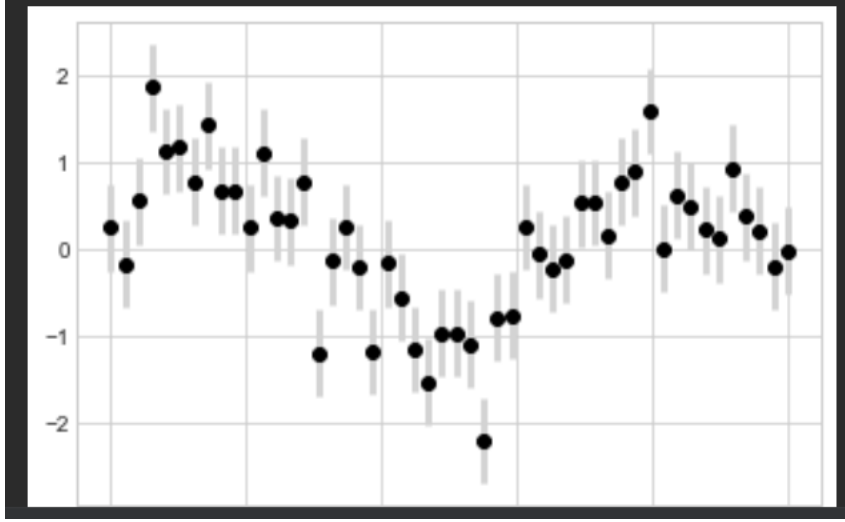
Visualize errors

```
plt.errorbar(x, y, yerr=dy, fmt='.r');
```

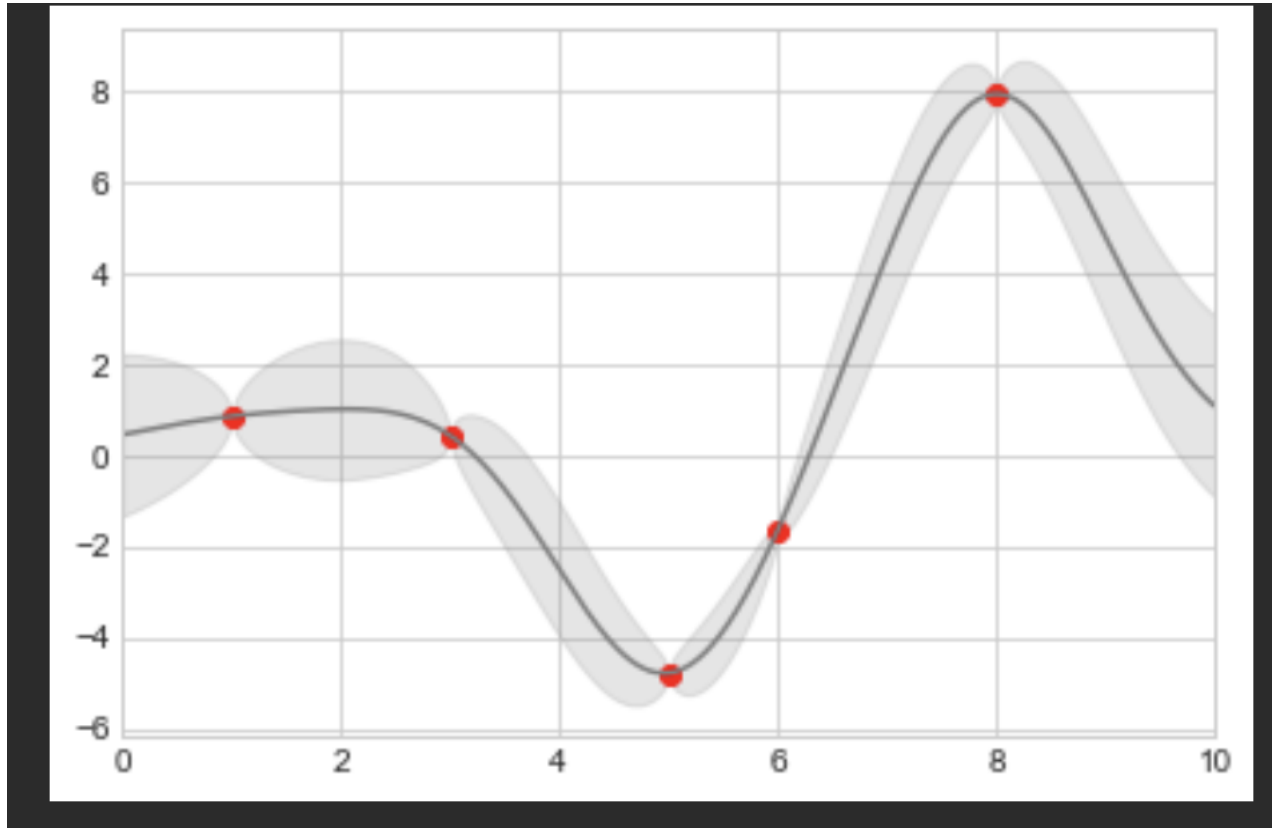


- The errorbar function has many options to fine-tune the outputs

```
plt.errorbar(x, y, yerr=dy, fmt='o', color='black',
             ecolor='lightgray', elinewidth=3, capsize=0);
```



- Continuous errorbar

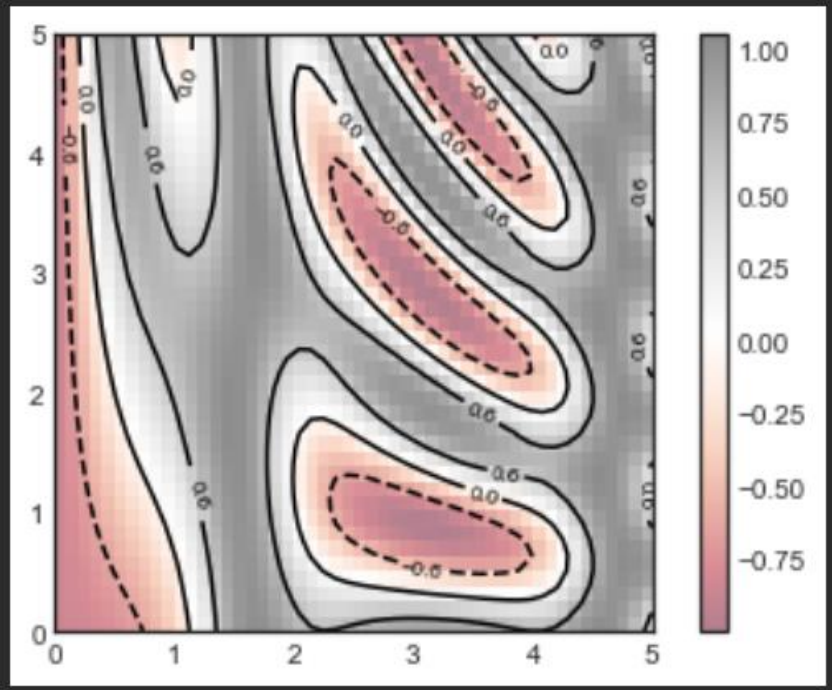


1.5. Density and Contour Plots

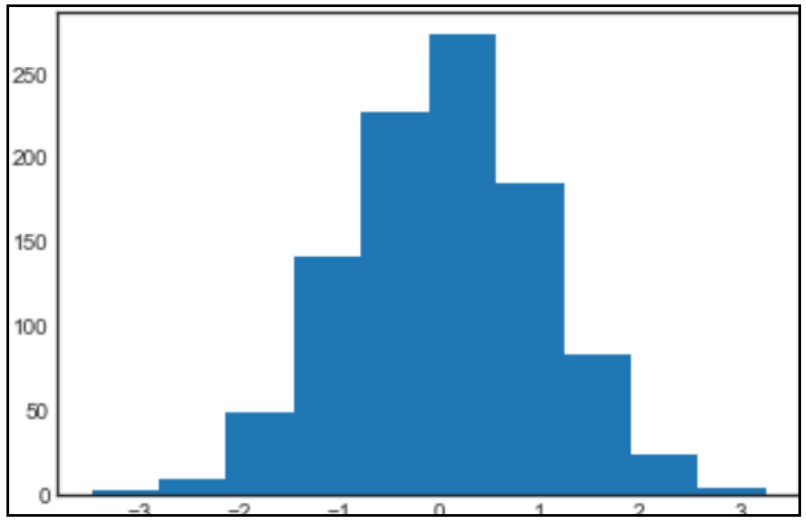
- Three Matplotlib functions to display three-dimensional data in two dimensions
  - plt.contour for contour plots
  - plt.contourf for filled contour plots
  - plt.imshow for showing images

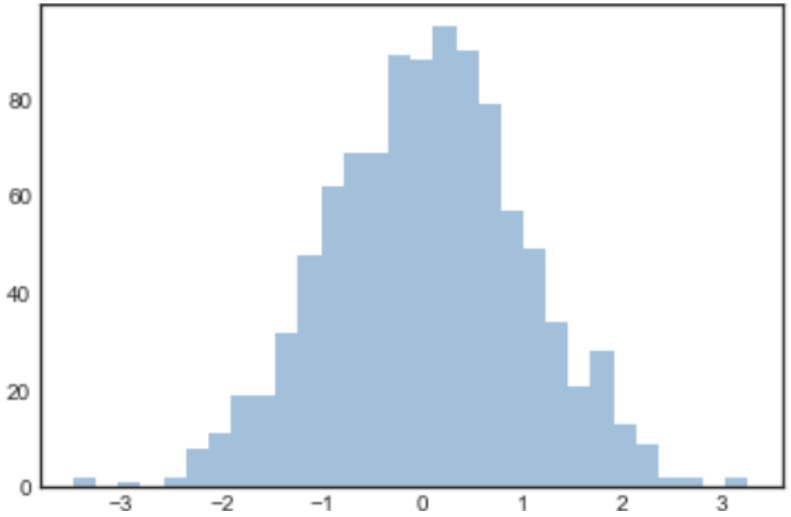
```python
# contours
contours = plt.contour(X, Y, Z, 3, colors='black')
plt.clabel(contours, inline=True, fontsize=8)

plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower',
           cmap='RdGy', alpha=0.5)
plt.colorbar();
```

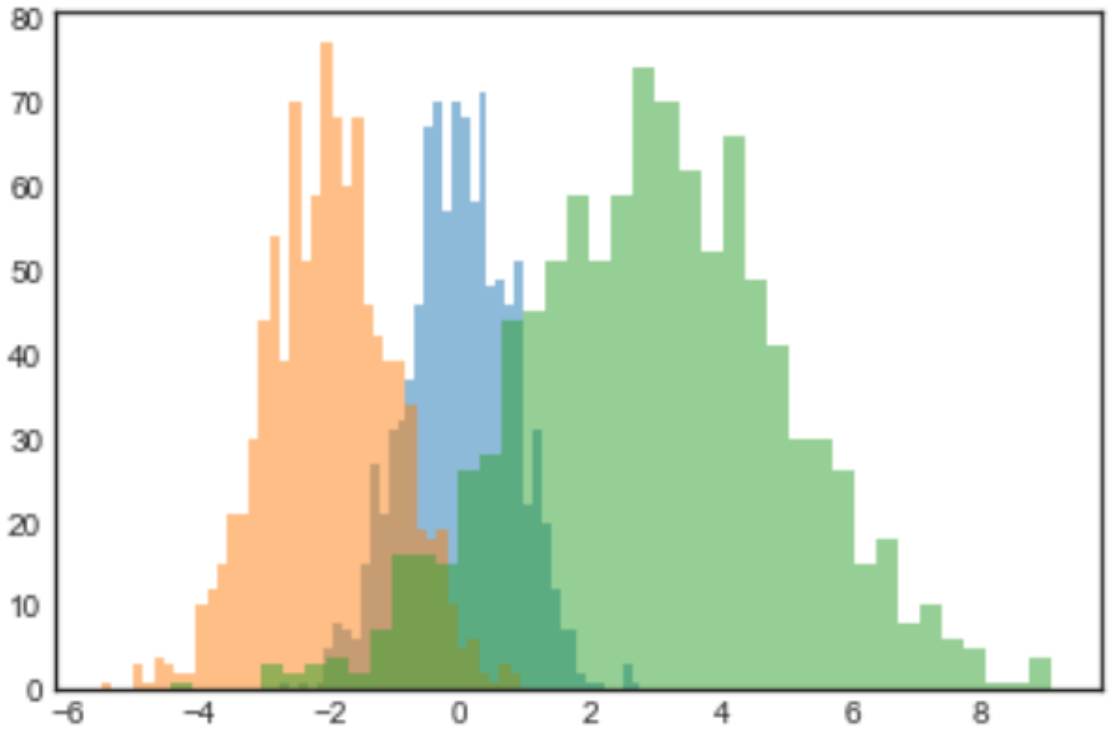


1.6. Histograms



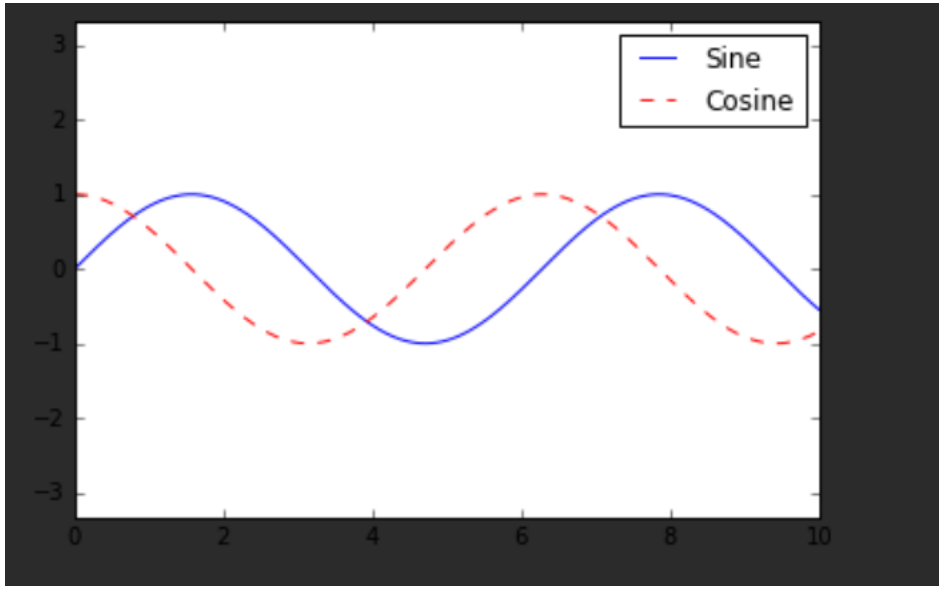


How to apply them?



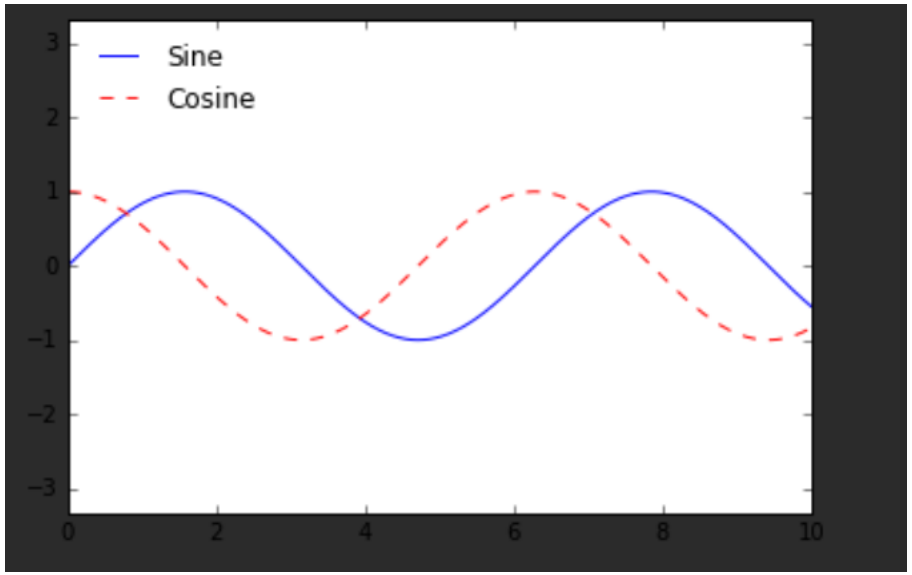
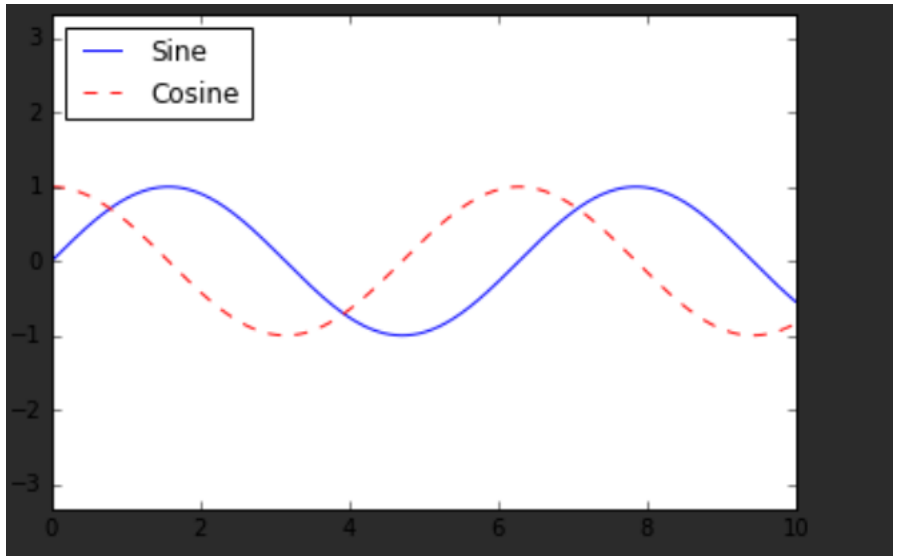
1.7. Customizing Plot Legends

- Plot legends give meaning to a visualization, assigning meaning to the various plot elements
- The simplest legend can be created with the plt.legend() command



Loc and frameon

ax.legend(loc='upper left', frameon=False)



ax.legend(loc='upper left', frameon=True)